

Proposta de Projeto: SignConnect LGP

Sistema Sensorial de Tradução de Linguagem Gestual Portuguesa com TinyML

Unidade Curricular: Competências Transferíveis II: MICSA

Março 2026

1 Introdução e Contextualização

A barreira de comunicação entre a comunidade surda e a população ouvinte em Portugal continua a ser um desafio significativo de inclusão social. Em paralelo, a convergência entre a microeletrónica de baixo consumo e a inteligência artificial tem catalisado o desenvolvimento de interfaces homem-máquina imersivas.

Este projeto propõe o desenvolvimento do **SignConnect LGP**, um protótipo de uma *smart glove* (luva inteligente) capaz de traduzir o alfabeto dactilológico da Linguagem Gestual Portuguesa (LGP) em dados digitais e texto. A inovação central da nossa abordagem reside na combinação de sensores *custom-made* de baixo custo (baseados em polímeros piezoresistivos) com algoritmos de *Machine Learning* embarcado (*TinyML*). Esta arquitetura permite que o reconhecimento dos gestos seja processado localmente no microcontrolador (*Edge Computing*), garantindo baixa latência e total independência de processamento na nuvem.

2 Objetivos

O projeto visa a conceção, montagem e programação de um sistema *wearable* completo, com os seguintes objetivos específicos:

- Desenvolver sensores de flexão customizados com *Velostat* para capturar a cinemática dos dedos.
- Integrar a leitura destes sensores analógicos e de um sensor inercial (IMU) num microcontrolador STM32F411RE, aplicando conceitos de conversão A/D e temporização determinística.
- Treinar e otimizar um modelo de rede neuronal (*TinyML*) capaz de classificar letras estáticas e movimentos simples da LGP, operando dentro das restrições de memória (RAM/Flash) do hardware.
- Transmitir a classificação final via Bluetooth para um terminal externo, apresentando a tradução em tempo real num *dashboard* web.

3 Arquitetura de Hardware e Materiais

A escolha dos componentes focou-se no compromisso entre a precisão necessária para a aquisição de sinais biométricos, a capacidade de processamento para inferência de *Machine Learning* e a viabilidade económica do protótipo.

Componente	Função e Justificação no Projeto
STM32F411RE	Microcontrolador central (ARM Cortex-M4). Escolhido pelo seu módulo ADC de 12-bits (essencial para ler os sensores de Velostat com precisão) e pela FPU integrada, crucial para acelerar os cálculos matriciais da rede neuronal.
Giroscópio (MPU6050)	Sensor inercial (IMU) com comunicação I2C. Fornece dados de aceleração e giroscópio nos 3 eixos, essenciais para detetar a orientação global da mão e gestos em movimento.
Folha de Velostat	Material piezoresistivo que altera a sua resistência elétrica quando dobrado. Constitui o núcleo ativo dos sensores de flexão a colocar em cada dedo da luva.
Fita de Dessoldar	Utilizada como eletrodo flexível (cobre trançado) nas faces do Velostat, garantindo condutividade sem comprometer a ergonomia e flexibilidade do sensor.
Módulo HC-05	Módulo Bluetooth Clássico para comunicação sem fios bidirecional (via protocolo UART) com o PC/Dashboard.
Luva de Ciclismo	Estrutura de suporte ergonómica, escolhida pela sua resiliência e facilidade de costura/fixação da cablagem e sensores.

Tabela 1: Especificação detalhada dos componentes de hardware.

3.1 Metodologia de Construção dos Sensores de Flexão

A construção dos sensores requer uma metodologia de "sanduíche condutora":

1. Corte do material piezoresistivo (*Velostat*) em tiras à medida das falanges dos dedos.
2. Aplicação da fita de dessoldar em ambas as faces do *Velostat*. O uso deste material evita a quebra dos condutores com a fadiga mecânica contínua da flexão.
3. Selagem do conjunto com fita isoladora retrátil para evitar curtos-circuitos e fixação mecânica à luva de ciclismo.

4 Metodologia de Software e Integração (Foco MICSA)

O desenvolvimento de software no ambiente *STM32CubeIDE* será estruturado de forma a aplicar os conceitos fundamentais da disciplina, dividindo-se nas seguintes etapas arquiteturais:

4.1 Aquisição Sensorial (ADC e Filtros)

Os sensores de *Velostat* serão implementados através de um circuito divisor de tensão. O pino central será ligado aos canais analógicos (ADC) do STM32. Para mitigar o ruído elétrico ("jitter") inerente a sensores resistivos caseiros e à interferência eletromagnética (EMI), será implementado um filtro digital de *Moving Average* (Média Móvel) via software para estabilizar as leituras antes de serem passadas ao modelo preditivo.

4.2 Sincronização e Timers Determinísticos

A precisão de um modelo de *Machine Learning* depende de uma taxa de amostragem (*Sample Rate*) estrita. O recurso a funções de *delay()* bloqueantes (*Polling*) será evitado. Em vez disso,

será configurado um **Hardware Timer** no STM32 para gerar interrupções a uma frequência fixa de **50Hz** (a cada 20ms). Na rotina de interrupção (ISR), o sistema fará a leitura síncrona do ADC e do I2C (MPU6050), garantindo um sinal biométrico temporalmente coerente.

4.3 Processamento Edge (TinyML) e Pipeline de Dados

O projeto adotará o fluxo de *Design Science Research* para o modelo de Inteligência Artificial:

1. **Criação do Dataset:** O STM32 enviará os dados em bruto (RAW) dos sensores para o PC via Serial, onde serão rotulados de acordo com o alfabeto LGP.
2. **Treino do Modelo:** Utilizaremos a plataforma *Edge Impulse* para extrair *features* e treinar uma Rede Neuronal artificial ligeira.
3. **Inferência no Edge:** O modelo treinado será exportado como uma biblioteca C++ otimizada e compilado juntamente com o firmware do STM32. A partir deste momento, a luva é capaz de inferir a letra autonomamente.

4.4 Apresentação e Interface Web

Para não saturar o barramento Bluetooth, o STM32 apenas transmitirá, via UART, o *label* da letra reconhecida (ex: GESTO: A, CONF: 95%). Um *script* local no PC (em *Node.js* ou *Python*) fará a leitura da porta COM virtual e atualizará dinamicamente uma página Web (Dashboard HTML/JS), providenciando *feedback* visual imediato ao utilizador.

5 Desafios Técnicos e Mitigação de Risco

O planeamento do projeto antecipa os seguintes desafios clássicos da engenharia de sistemas embebidos, com as devidas estratégias de resolução:

- **Ruído Inercial (Drift):** A integração matemática dos dados do giroscópio gera erros acumulados ao longo do tempo. **Solução:** Ativação e utilização do *Digital Motion Processor* (DMP) integrado no MPU6050 para obter as rotações estabilizadas em ângulos de Euler/Quaternions diretamente do hardware.
- **Latência de Comunicação (UART):** A transmissão bloqueante via Bluetooth pode violar o tempo estrito da amostragem a 50Hz. **Solução:** Configuração da UART no STM32 com *Direct Memory Access* (DMA), libertando o CPU principal dessa tarefa.
- **Limitações de Memória RAM (TinyML):** Modelos de rede neuronal podem causar esgotamento de memória (*HardFault*). **Solução:** Utilização do compilador *EON* da plataforma *Edge Impulse* (ou CMSIS-NN), que otimiza as matrizes matemáticas para poupar até 30% da RAM do microcontrolador.

6 Cronograma Preliminar

O desenvolvimento será iterativo, distribuído em 4 fases fundamentais:

1. **Fase 1:** Construção física da luva, acondicionamento dos sensores de *Velostat* e testes de leitura simples de ADC e I2C (Debugging).
2. **Fase 2:** Implementação dos *Timers*, captura do dataset inicial de gestos e envio para o *Edge Impulse*.

3. **Fase 3:** Treino do modelo, integração da biblioteca gerada no STM32CubeIDE e otimização de RAM.
4. **Fase 4:** Estabelecimento da comunicação Bluetooth, criação do *Dashboard* Web final e testes de validação com utilizadores.

7 Referências Bibliográficas

A proposta suporta-se na bibliografia da cadeira e na arquitetura de soluções validadas pela comunidade académica:

- Martins, R. E., *Sebentas da U.C. de MICA: Sensores e Actuadores, Microcontroladores, ADC, Timers e Debugging*, Universidade de Aveiro, 2025/2026.
- STMicroelectronics, *UM2553 - STM32CubeIDE quick start guide* e *Datasheet STM32F411RE*.
- *Smart-Glove Engineering Project* (Repositório GitHub: ghimireadarsh/Smart-Glove).
- *Embedded machine learning for gesture recognition* (Repositório GitHub: devnithw/gesture-tinyml).